SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 - Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	192424069
Student Name	Aditya Krishnan S

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute 10^8 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform: ShopSphere is a growing e-commerce platform whose product catalogue currently holds n = 100,000 products and is expected to reach n = 1,000,000 within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

Module M1 – Duplicate-Listing Detector: Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n − 2 DO
  FOR j ← i + 1 TO n − 1 DO
    IF Product[i].title = Product[j].title AND Product[i].seller = Product[j].seller THEN
      MarkDuplicate(j)
    END IF
  END FOR
END FOR
```

Module M2 – API Rate Limiter: The public API audits request windows of doubling size using the routine below. A team member claims that it is Θ(n log n) because the outer loop runs about log n times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
  FOR j ← 0 TO i − 1 DO
    Audit(Request[j])          // basic operation
  END FOR
  i ← i × 2
END WHILE
```

Module M3 – Product Search: Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by comparing the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies T(n) = T(n/2) + 1, T(1) = 1.

Module M4 – Recommendation Engine: The engine splits the customer–product data into four parts of size n/2 (because of replicated partitions) and merges the partial results in linear time: T(n) = 4T(n/2) + n. A proposed redesign removes the replication, giving T(n) = 2T(n/2) + n.

Module M5 – Sales Analytics Engine: The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving $T(n) = 2T(n/2) + n \log n$. Separately, profiling shows that the nightly report generator performs $f(n) = 3n^2 + 10n \log_2 n + 50$ operations.

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

(b) Analyse the Iterative Modules: For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's $\Theta(n \log n)$ claim is correct and give the tight bound. Verify your M2 result for n = 1,024.

(c) Solve a Recurrence by Substitution: Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

(d) Apply the Master Theorem: For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a, b, f(n), n^(log_b a) and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to $T(n) = 2T(n/2) + n \log n$; if not, derive the complexity using the extended Master Theorem or a recursion tree.

(e) Analyse Best, Average and Worst Cases: For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p, and evaluate it for n = 1,000,000 and p = 0.5.

(f) Prove an Asymptotic Bound: Using the formal definitions, prove that the report generator's cost $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$ by finding constants c_1, c_2 and n_0, and show that f(n) is not O(n).

(g) Scalability Decision: Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for n = 1,000,000. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Mark Evaluation:

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20				
Application of Concepts	30				
Problem-Solving Approach	20				
Accuracy of Solution	20				
Clarity	10				
Total Marks					

Code of Conduct

I ADITYA KRISHNAN S (192424069), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:



Date: 12 October 2026

(A) UNDERSTAND THE PROBLEM: MODULE BREAKDOWN

Module M1 (Duplicate-Listing Detector):

→ Input Size (n): Total number of products in the catalogue (n).
→ Basic Operation: String comparison (Product[i].title = Product[j].title AND Product[i].seller = Product[j].seller).
→ Occurrence: Inside a nested loop comparing every distinct pair of products (i, j) with $j > i$.
→ Growth & Justification: Grows quadratically ($\Theta(n^2)$) because the number of pairs checked grows as $n(n-1)/2$.

Module M2 (API Rate Limiter):

→ Input Size (n): Number of requests in the current window.
→ Basic Operation: Audit(Request[j]).
→ Occurrence: Inside the inner loop executing i times for each doubling step of i.
→ Growth & Justification: Grows linearly-logarithmically ($\Theta(n)$) because the total operations sum to $1 + 2 + 4 + \cdots + n/2 = n - 1$.

Module M3 (Product Search):

→ Input Size (n): Number of indexed products.
→ Basic Operation: Key comparison against an array element/middle entry.
→ Occurrence: Sequential scan (S1) or binary partitioning (S2).
→ Growth & Justification: S1 grows linearly ($\Theta(n)$) due to sequential scanning; S2 grows logarithmically ($\Theta(\log n)$) because the search space is halved at each step.

Module M4 (Recommendation Engine):

→ Input Size (n): Size of the customer-product dataset.
→ Basic Operation: Data partition merging/combination steps.
→ Occurrence: Recursive divide-and-conquer splitting.
→ Growth & Justification: Current design ($4T(n/2) + n$) grows as $\Theta(n^2)$; proposed design ($2T(n/2) + n$) grows as $\Theta(n \log n)$.

Module M5 (Sales Analytics Engine):

→ Input Size (n): Number of sales records.
→ Basic Operation: Revenue comparisons/sorting operations and polynomial arithmetic evaluation.
→ Occurrence: Divide-and-conquer sorting merge levels and nightly polynomial computation.
→ Growth & Justification: Recurrence $T(n) = 2T(n/2) + n \log n$ grows as $\Theta(n \log^2 n)$, and the report generator function $f(n) = 3n^2 + 10n \log_2 n + 50$ grows as $\Theta(n^2)$.

(B) ANALYSE THE ITERATIVE MODULES

Module M1: Exact Comparisons and Complexities
Exact Number of Comparisons:
The outer loop runs from $i = 0$ to $n - 2$, and the inner loop runs from $j = i + 1$ to $n - 1$. The total number of comparisons is:

$$\sum_{i=0}^{n-2} (n - 1 - i) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}$$

Time Complexity: $\Theta(n^2)$
Space Complexity: $\Theta(1)$ auxiliary space (in-place execution).

Module M2: Audit Operations & Team Member Claim Analysis
Exact Number of Operations:
The outer loop variable i takes values 1,2,4,8, …, $2^k < n$.
For a given i, the inner loop runs i times. Total operations $T(n)$:

$$T(n) = \sum_{j=0}^{\log_2 n - 1} 2^j = 1 + 2 + 4 + \cdots + \frac{n}{2} = n - 1$$

Claim Verification: The team member's claim that it is $\Theta(n \log n)$ is **incorrect**. The sum of a geometric series whose terms double each step up to $n/2$ equals $n - 1$, making the tight asymptotic bound $\Theta(n)$ (Linear time).
Verification for $n = 1{,}024$ (2^{10}):

$$T(1024) = 1024 - 1 = 1{,}023 \text{ operations}$$

(C) SOLVE A RECURRENCE BY SUBSTITUTION

For search version S2: $T(n) = T\left(\frac{n}{2}\right) + 1$, with base case $T(1) = 1$.

Backward Substitution Steps:

$T(n) = T\left(\frac{n}{2}\right) + 1$

Substitute $T\left(\frac{n}{2}\right) = T\left(\frac{n}{4}\right) + 1$:

$T(n) = \left[T\left(\frac{n}{4}\right) + 1\right] + 1 = T\left(\frac{n}{4}\right) + 2$

Substitute $T\left(\frac{n}{4}\right) = T\left(\frac{n}{8}\right) + 1$:

$T(n) = T\left(\frac{n}{8}\right) + 3$

Generalizing to the k-th substitution step:

$T(n) = T\left(\frac{n}{2^k}\right) + k$

Reach the base case when $\frac{n}{2^k} = 1 \Longrightarrow k = \log_2 n$.

Substitute $k = \log_2 n$ and $T(1) = 1$:

$T(n) = T(1) + \log_2 n = 1 + \log_2 n$

Closed Form: $T(n) = \log_2 n + 1$

Complexities:
Iterative Implementation: Time Complexity = $\Theta(\log n)$, Space Complexity = $\Theta(1)$.
Recursive Implementation: Time Complexity = $\Theta(\log n)$, Space Complexity = $\Theta(\log n)$ (due to the recursive call stack).

(D) APPLY THE MASTER THEOREM

Module M4: Recommendation Engine

Current Design:

$T(n) = 4T\left(\frac{n}{2}\right) + n$

Identify parameters: $a = 4$, $b = 2$, $f(n) = n$.
Calculate $n^{\log_b a} = n^{\log_2 4} = n^2$.
Compare $f(n)$ with $n^{\log_b a}$: $n = O(n^{2-\epsilon})$ for $\epsilon = 1$.

Master Theorem Case Used:
Case 1.

Result:
$T(n) = \Theta(n^2)$. When data doubles ($2n$), running time increases by a factor of $2^2 = 4$.

Proposed Redesign:

$T(n) = 2T\left(\frac{n}{2}\right) + n$

Identify parameters: $a = 2$, $b = 2$, $f(n) = n$.
Calculate $n^{\log_b a} = n^{\log_2 2} = n^1 = n$.
Compare $f(n)$ with $n^{\log_b a}$: $f(n) = \Theta\left(n^{\log_b a}\right) = \Theta(n)$.

Master Theorem Case Used:
Case 2.

Result: $T(n) = \Theta(n \log n)$. When data doubles ($2n$), running time increases by roughly $2\times$ plus a log factor.

Module M5: Sales Analytics Recurrence

Recurrence:

$T(n) = 2T\left(\frac{n}{2}\right) + n \log n$

Identify parameters: $a = 2, b = 2, f(n) = n \log n$.

Calculate $n^{\log_b a} = n^{\log_2 2} = n$.

Check standard cases: $f(n) = n \log n$ is polynomially larger than $n^{\log_b a} = n$ because $\frac{n \log n}{n} = \log n$, which grows faster than any polynomial bound n^{ϵ} for small ϵ. Thus, standard Master Theorem Case 3 fails.

Extended Master Theorem / Recursion Tree Derivation:

Using a recursion tree, at level j (from 0 to $\log_2 n - 1$), there are 2^j subproblems, each of size $\frac{n}{2^j}$.

Work done at level j: $2^j \times \left(\frac{n}{2^j} \log\left(\frac{n}{2^j}\right)\right) = n(\log n - j)$.

Summing across all $\log_2 n$ levels:

$$T(n) = \sum_{j=0}^{\log_2 n - 1} n\,(\log n - j) = n \sum_{k=1}^{\log_2 n} k = \Theta(n \log^2 n)$$

(E) ANALYSE BEST, AVERAGE AND WORST CASES FOR SEARCH

Search Version S1 (Sequential Scan):

- → **Best Case:** The product is found at the first position ($i = 0$). Comparisons = 1.
- → **Worst Case:** The product is at the last position or not present. Comparisons = n.
- → **Average Case:** Assuming uniform distribution where the target exists with probability p:
  - If it exists (probability p), the expected position is $\frac{n+1}{2}$, taking $\frac{n+1}{2}$ comparisons.
  - If it does not exist (probability $1 - p$), it takes n comparisons.
- → **Average Comparisons:** $A(n) = p\left(\frac{n+1}{2}\right) + (1-p)n$.
- → **Evaluation for $n = 1{,}000{,}000$ and $p = 0.5$:**
- → $A(1{,}000{,}000) = 0.5\left(\frac{1{,}000{,}000+1}{2}\right) + (0.5)(1{,}000{,}000) = 250{,}000.25 + 500{,}000 = 750{,}000.25$ comparisons

Search Version S2 (Binary Search):

- → **Best Case:** Target is at the exact middle index on the first check. Comparisons = 1.
- → **Worst Case:** Target is at the leaf level or absent. Comparisons = $\lfloor \log_2 n \rfloor + 1$.
- → **Average Case:** Expected number of comparisons for a successful search is approximately $\log_2 n - 1$.

(F) PROVE AN ASYMPTOTIC BOUND

Prove $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$:

We must find positive constants c_1, c_2, and n_0 such that $c_1 n^2 \le 3n^2 + 10n \log_2 n + 50 \le c_2 n^2$ for all $n \ge n_0$.

Lower Bound ($\Omega(n^2)$): For all $n \ge 1$, dropping lower-order positive terms:

$$3n^2 + 10n \log_2 n + 50 \ge 3n^2$$

Thus, choosing $c_1 = 3$ and $n_0 = 1$ satisfies $3n^2 \le f(n)$.

Upper Bound ($O(n^2)$): For $n \ge 1$, since $\log_2 n \le n$:

$$3n^2 + 10n \log_2 n + 50 \le 3n^2 + 10n(n) + 50n^2 = (3 + 10 + 50)n^2 = 63n^2$$

Thus, choosing $c_2 = 63$ and $n_0 = 1$ satisfies $f(n) \le 63n^2$.

Since both bounds hold, $f(n) = \Theta(n^2)$.

Show $f(n)$ is not $O(n)$:

Suppose by contradiction that $f(n) = O(n)$. Then there exist constants $C > 0$ and n_0 such that $3n^2 + 10n \log_2 n + 50 \le Cn$ for all $n \ge n_0$.

Dividing both sides by n:

$$3n + 10\log_2 n + \frac{50}{n} \leq C$$

As $n \to \infty$, the left-hand side grows without bound (∞), which can never be less than or equal to the constant C. Therefore, $f(n)$ is **not** $O(n)$.

(G) SCALABILITY DECISION ($n = 1,000,000$)

Server Capacity = 10^8 operations/sec. Target Nightly Window = 10 minutes $= 600$ seconds.
Maximum allowable operations = $600 \times 10^8 = 6 \times 10^{10}$ operations.

Module	Asymptotic Complexity	Operations for n=1,000,000	Estimated Runtime (at 108 ops/sec)	Meets 10-Min Window? (<6×1010 ops)
M1 (Duplicate Detector)	$\Theta(n^2)$	$\frac{10^6(10^6-1)}{2} \approx 5 \times 10^{11}$	5,000 seconds ≈ 83.3 minutes	**FAIL**
M2 (API Rate Limiter)	$\Theta(n)$	$1{,}000{,}000 - 1 \approx 10^6$	0.01 seconds	**PASS**
S2 (Binary Search Index)	$\Theta(\log n)$	$\log_2(10^6) + 1 \approx 21$	< 0.0001 seconds	**PASS**
M4 (Current Design)	$\Theta(n^2)$	$(10^6)^2 = 10^{12}$	10,000 seconds ≈ 166.7 minutes	**FAIL**
M4 (Proposed Design)	$\Theta(n \log n)$	$10^6 \log_2(10^6) \approx 1.99 \times 10^7$	0.2 seconds	**PASS**
M5 (Analytics Engine)	$\Theta(n \log^2 n)$	$10^6 \times (\log_2 10^6)^2 \approx 3.96 \times 10^8$	3.96 seconds	**PASS**
M5 (Report Generator)	$\Theta(n^2)$	$3(10^6)^2 + \cdots \approx 3 \times 10^{12}$	30,000 seconds ≈ 500 minutes	**FAIL**

Recommendations & Redesign Strategy:

1. **Module M1 (Duplicate-Listing Detector): Must be redesigned.** Replace the $O(n^2)$ brute-force double loop with a hashing-based approach or sorting keys lexicographically by title and seller, reducing complexity to $\Theta(n \log n)$ and running time to fractions of a second.
2. **Module M4 (Recommendation Engine): Adopt the proposed redesign immediately.** Shifting from $4T(n/2) + n$ ($\Theta(n^2)$) to $2T(n/2) + n$ ($\Theta(n \log n)$) cuts operations from 10^{12} down to 1.99×10^7, comfortably passing the scalability threshold.
3. **Module M5 (Report Generator): Must be redesigned.** The polynomial function $f(n) = 3n^2 + \cdots$ scales quadratically and violates the window limit at $n = 1{,}000{,}000$. Pre-aggregating data incrementally or utilizing parallel stream processing is required to avoid batch processing bottlenecks.
